

Sprawozdanie z laboratoriów z przedmiotu Sieci Neuronowe

Implementacja i analiza perceptronu wielowarstwowego w NumPy

Aleksander Brandt

Inżynieria i Analiza Danych, III rok

19 kwietnia 2026 r.

Spis treści

1	Zakres pracy	1
2	Metodyka	2
3	Walidacja implementacji i wpływ architektury	2
3.1	Forward pass i pojemność modelu	2
3.2	Backpropagation i rozmiar batcha	3
4	Optymalizacja uczenia	4
5	Warstwa wyjściowa i zadania klasyfikacyjne	4
6	Funkcje aktywacji i pojemność modelu	5
7	Przeuczenie i regularyzacja	7
8	Wnioski	9

Streszczenie

W raporcie opisuję własną implementację MLP w NumPy i wyniki wybranych eksperymentów z siedmiu tygodni laboratorium. Układ jest tematyczny, a nie chronologiczny: wyniki grupuję według tego, co pokazują – weryfikację implementacji, wpływ architektury, optymalizatora, aktywacji i regularyzacji – a nie według numeru kolejnych zajęć. Pomijam zbiory, na których wyniki różnią się nieistotnie, i skupiam się na tych przypadkach, w których widać wyraźną różnicę między ustawieniami.

1 Zakres pracy

W sprawozdaniu zestawiam wyniki własnej implementacji MLP i wnioski z eksperymentów. Pokazuję kolejno: weryfikację *forward pass* względem PyTorch, własny backpropagation, porównanie Momentum z RMSProp, Softmax na warstwie wyjściowej, dobór funkcji aktywacji oraz regularyzację ℓ_2 i wczesne zatrzymanie.

Nie powtarzam wszystkich przebiegów z notebooków. Dla każdego tematu wybieram jeden lub dwa przypadki, które faktycznie coś zmieniają w wynikach, i na nich buduję wnioski. Tam, gdzie zmiana hiperparametru nie miała widocznego efektu (np. najprostsze zbiory w NN1), nie zajmuję miejsca na opis.

2 Metodyka

Wszystko, poza NN1, liczyłem we własnym kodzie opartym tylko na NumPy – propagację wstecz, aktualizacje wag, mini-batche, Momentum, RMSProp, Softmax, wybór aktywacji i regularyzację. PyTorch użyłem tylko w NN1 jako źródło wag do walidacji *forward pass*.

Dane wejściowe skalowałem min-max (w NN1–NN2) lub standaryzowałem (w późniejszych eksperymentach), zawsze dopasowując transformację do zbioru treningowego. Wszystkie metryki w tabelach są liczone na skali oryginalnej. Dla regresji raportuję

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

a dla klasyfikacji F_1^{macro} , bo na zbiorach wieloklasowych jest bardziej informatywne od zwykłej trafności.

Przy strojeniu uruchamiałem eksperymenty na kilku seedach, ale w notebookach zapisany jest zwykle jeden reprezentatywny przebieg (seed 42 lub 20260324). Kolejność wyników wśród porównywanych wariantów była powtarzalna między seedami, dlatego w raporcie pokazuję pojedyncze przebiegi.

Warto zaznaczyć, że NN3 nie jest czystym A/B. Porównuję tam najlepsze znalezione konfiguracje Momentum i RMSProp, nie te same architektury przy tym samym η . Wzięło się to stąd, że w poleceniu NN3 były narzucone progi MSE (1 na **square-large**, 6 na **steps-large**, 9 na **multimodal-large**) i stroilem oba optymalizatory osobno, tak żeby dla każdego z nich maksymalnie zbliżyć się do celu. Wniosek z tej sekcji rozumiem więc jako „który optymalizator łatwiej dał się doprowadzić do dobrego wyniku”, a nie jako różnicę wynikającą wyłącznie ze wzoru aktualizacji.

3 Walidacja implementacji i wpływ architektury

3.1 Forward pass i pojemność modelu

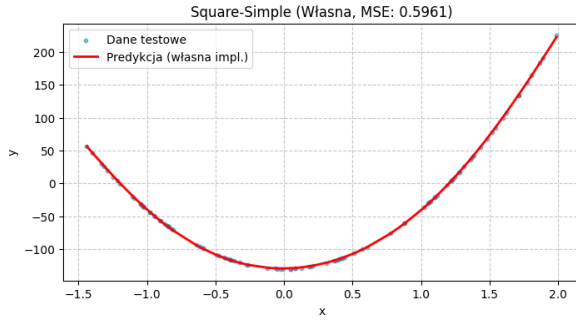
Zacząłem od zbudowania MLP dla regresji jednowymiarowej i sprawdzenia, czy mój *forward pass* daje te same liczby, co PyTorch po nauczaniu tej samej sieci. Przy okazji widać, jak sama pojemność wpływa na jakość aproksymacji.

Tabela 1: NN1: wyniki modeli uczonych w PyTorchu, raportowane na nieznormalizowanym zbiorze testowym.

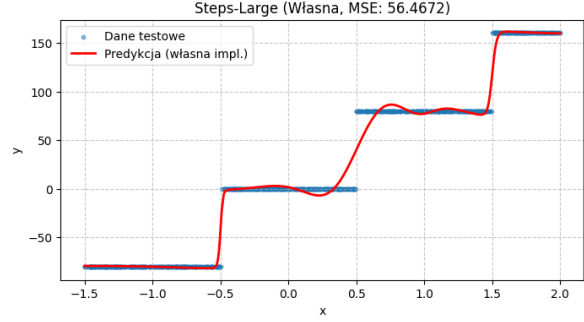
Zbiór	Architektura	Liczba kroków	MSE _{test}
square-simple	1-5-1	15000	0,2281
square-simple	1-10-1	15000	0,8845
square-simple	1-5-5-1	20000	0,5961
steps-large	1-5-1	25000	312,0022
steps-large	1-10-1	25000	162,6072
steps-large	1-5-5-1	30000	56,4672

Na **square-simple** wygrywa najprostsza sieć 1-5-1. Funkcja jest gładka i jednowymiarowa, więc większa pojemność tylko utrudnia strojenie. Na **steps-large** jest odwrotnie – liczy się zdolność złożenia kilku stromych fragmentów w jeden kształt. Przejście z 1-5-1 na 1-10-1 trochę pomaga, ale dopiero druga warstwa ukryta daje wyraźny skok jakości. Dla funkcji schodkowych bardziej oplaca się zwiększanie głębokości niż szerokości.

Najważniejsze na tym etapie było jednak coś innego: po skopiowaniu wag z PyTorch do własnego kodu dostałem dokładnie te same MSE dla 1-5-5-1, czyli 0,5961 i 56,4672. To potwierdza, że kolejność mnożeń, dodawanie biasów i wyjście liniowe są zgodne z referencją.



(a) **square-simple**: odpowiedź własnej implementacji po przeniesieniu wag.



(b) **steps-large**: wynik własnej implementacji dla najlepszej architektury.

Rysunek 1: Walidacja *forward pass* względem modelu referencyjnego w PyTorchu.

3.2 Backpropagation i rozmiar batcha

Po walidacji *forward pass* zaimplementowałem pełną propagację wsteczną. Sieć trzyma aktywacje pośrednie, liczy gradienty względem wag i biasów, a wagi aktualizuje po pełnym zbiorze, po mini-batchach albo po bardzo małych porcjach.

Tabela 2: NN2: wyniki własnego backpropagation na trzech zbiorach.

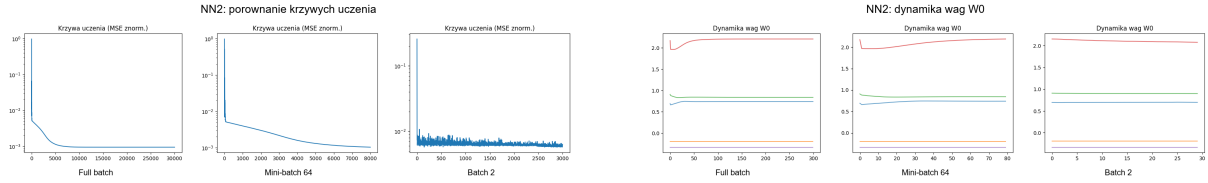
Zbiór	Architektura	Batch	γ	Epoki	MSE_{test}
square-simple	1-32-1	8	0,9	1500	2,6260
steps-small	1-32-32-1	2	0,0	8000	384,8258
multimodal-large	1-64-64-1	64	0,9	12000	24,8094

Na **square-simple** i **multimodal-large** próg z polecenia udało się spełnić. Najtrudniejszy okazał się **steps-small**: mało punktów treningowych i ostra funkcja schodkowa to kiepskie środowisko dla czystego spadku gradientowego – sieć potrafi się doskonale dopasować do treningu, ale w pustych obszarach osi x ustawia progi schodka losowo.

Tabela 3: NN2: wpływ rozmiaru batcha na **steps-small**, architektura 1-16-16-1.

Tryb uczenia	Batch	Epoki	MSE_{test}
pełnobatchowy	pełny	6000	161,2039
mini-batch	64	2500	160,6092
quasi-online	2	4000	413,9542

Miedzy wariantami zmieniała się też liczba epok i η , więc wniosku nie traktuję jako czystego A/B – ale kierunek jest czytelny. Mini-batch 64 był jednocześnie szybszy i trochę lepszy od pełnego batcha, a batch 2 okazał się za bardzo zaszumiony. Dla funkcji schodkowej taki szum nie pomaga, tylko rozrzuca progi przejścia po osi x .



(a) Krzywe uczenia dla pełnego batcha, mini-batcha 64 i batcha 2.

(b) Trajektorie wybranych wag pierwszej warstwy w tych samych trzech wariantach.

Rysunek 2: Wpływ rozmiaru batcha na stabilność uczenia w NN2.

4 Optymalizacja uczenia

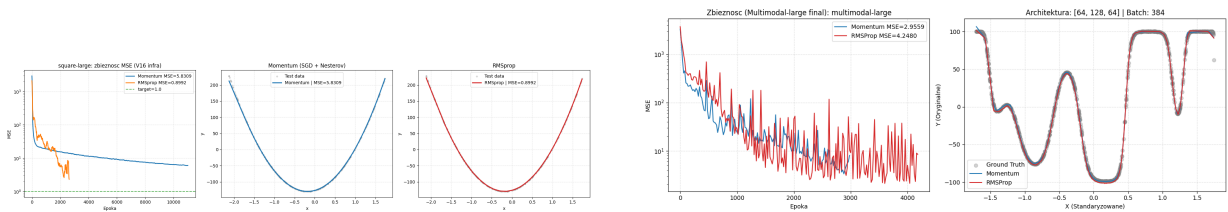
W kolejnym kroku porównywałem Momentum i RMSProp. Najciekawsza obserwacja nie dotyczy samego wzoru, tylko tego, jak łatwo dany optymalizator dał się doprowadzić do wyniku poniżej progu z polecenia.

Tabela 4: NN3: najlepsze znalezione konfiguracje dla obu optymalizatorów. Jak zaznaczyłem wyżej, to porównanie *best-found*, nie czysty A/B.

Zbiór	Optymalizator	Architektura	Aktywacja	Batch	η	Wynik końcowy
square-large	RMSProp	64-32	ReLU	32	0,0022	MSE = 0,8992 , best ep. 2630
square-large	Momentum	32	tanh	8	0,0068	MSE = 5,8309, best ep. 10980
multimodal-large	Momentum	64-128-64	ReLU	384	0,00220	MSE = 2,9559
multimodal-large	RMSProp	64-128-64	ReLU	384	0,00009	MSE = 4,2480
steps-large	RMSProp	96-64-48-32-24	leaky ReLU	192	0,00024	MSE = 4,6370
steps-large	Momentum	128-64-32	tanh	8	0,00190	MSE = 16,8323

Na **square-large** RMSProp schodzi poniżej progu około epoki 2630, a najlepsze Momentum zatrzymuje się na 5,83 mimo dłuższego treningu. Na **steps-large** różnica jest jeszcze większa – tylko RMSProp ma końcowe MSE mniejsze niż 6, Momentum stoi ponad 16. Przy ostrych przejściach funkcji pomaga adaptacyjne dopasowywanie długości kroku w różnych kierunkach.

Na **multimodal-large** wygrywa jednak Momentum. Funkcja celu jest tu gładka, ale ma wiele lokalnych ekstremów i rozpęd *pomaga* przejeżdżać przez płytkie minima. To wystarczy, żeby nie traktować RMSProp jako ogólnie lepszego algorytmu.



(a) **square-large**: najlepsze konfiguracje Momentum i RMSProp.

(b) **multimodal-large**: oba działają, wygrywa Momentum.

Rysunek 3: NN3: porównanie najlepszych wariantów.

5 Warstwa wyjściowa i zadania klasyfikacyjne

W części klasyfikacyjnej zaimplementowałem Softmax i odpowiednią modyfikację propagacji wstecznej. Porównanie Softmax vs sigmoidę przeprowadziłem w dwóch krokach: najpierw na tych samych hiperparametrach, żeby zobaczyć czystą różnicę warstwy wyjściowej, później z lekkim dostrojeniem finalistów.

Tabela 5: NN4: bazowe porównanie Softmax i sigmoidy na warstwie wyjściowej.

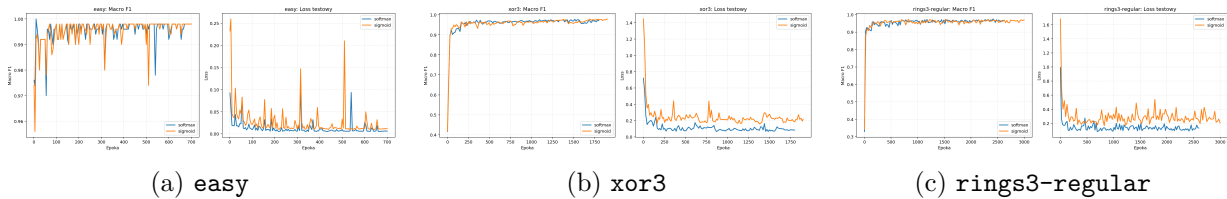
Zbiór	Wyjście	Architektura	Optymalizator	η	Najlepsze F_1^{macro}
easy	softmax	16-16	Momentum	0,0500	1,0000
easy	sigmoid	16-16	Momentum	0,0500	0,9980
xor3	softmax	32-32-16	Momentum	0,0300	0,9734
xor3	sigmoid	32-32-16	Momentum	0,0300	0,9713
rings3-regular	softmax	64-64-32	RMSProp	0,0030	0,9754
rings3-regular	sigmoid	64-64-32	RMSProp	0,0025	0,9722

Softmax wygrywa na wszystkich trzech zbiorach, ale przewaga jest mała. Tam, gdzie klasy są wzajemnie konkurencyjne (**rings3-regular**), probabilistyczna interpretacja wyjścia pomaga bardziej niż tam, gdzie problem jest łatwy (**easy**) i niemal każdy rozsądny model daje $F_1^{\text{macro}} \approx 1$.

Tabela 6: NN4: finaliści po lekkim dostrojeniu.

Zbiór	Wariant końcowy	Architektura	Epoki	Best F_1^{macro}	Final F_1^{macro}
easy	softmax_base	16-16	700	1,0000	0,9980
rings3-regular	softmax_base	64-64-32	2600	0,9754	0,9659
xor3	sigmoid_more_epochs	32-32-16	1900	0,9795	0,9795

Po dostrojeniu widać, że na xor3 sigmoidy wydłużona o kilka tysięcy epok dorzuca tyle, że wyprzedza Softmax. Softmax dawał lepszy punkt startowy, ale nie jest nie do ruszenia.



Rysunek 4: Krzywe F_1^{macro} dla Softmax i sigmoidy.

6 Funkcje aktywacji i pojemność modelu

W tej części przetestowałem sigmoid, tanh, ReLU i aktywację liniową na zbiorze **multimodal-large** dla kilku architektur, a później przeniósłem najlepsze konfiguracje na pozostałe zbiory. Wszystko w tym notebooku chodziło z RMSProp, więc wnioski dotyczą relacji *aktywacja-architektura*, a nie samego wyboru optymalizatora.

Tabela 7: NN5: agregacja wyników na **multimodal-large** po wszystkich badanych architekturach.

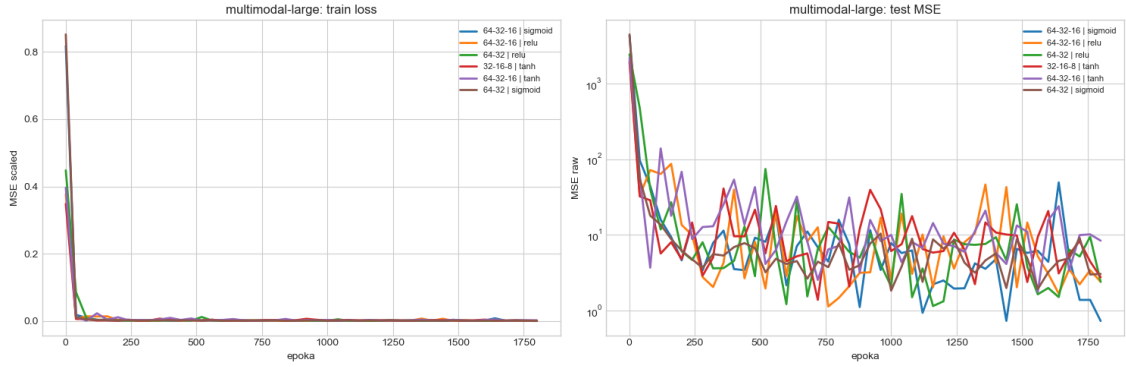
Aktywacja	Min. MSE	Śr. MSE	Maks. MSE	Odch. std.
sigmoid	0,726	220,78	1426,94	472,0
tanh	1,382	12,80	72,94	23,1
ReLU	1,128	458,48	1534,89	660,4
liniowa	4433,57	4433,60	4433,76	0,06

Sigmoid dał najlepszy pojedynczy wynik, ale miał ogromny rozrzut. tanh nie wygrał rekordem, za to był najbardziej stabilny między architekturami. ReLU było bardzo dobre w większych sieciach i bardzo słabe w mniejszych – stąd duże odchylenie. Aktywacja liniowa w warstwach ukrytych zwyczajnie nie działała, bo redukuje cały model do złożenia transformacji liniowych.

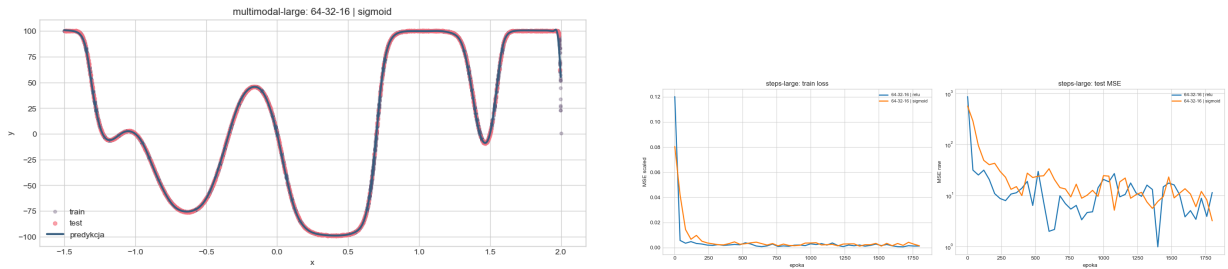
Tabela 8: NN5: najlepsze konfiguracje przeniesione na zbiory docelowe.

Zbiór	Architektura	Aktywacja	Metryka testowa
multimodal-large	64-32-16	sigmoid	MSE = 0,726
multimodal-large	64-32-16	ReLU	MSE = 1,128
steps-large	64-32-16	ReLU	MSE = 0,983
steps-large	64-32-16	sigmoid	MSE = 3,181
rings3-regular	64-32-16	sigmoid	$F_1^{\text{macro}} = \mathbf{0,973}$
rings3-regular	64-32-16	ReLU	$F_1^{\text{macro}} = 0,970$
rings5-regular	64-32-16	ReLU	$F_1^{\text{macro}} = \mathbf{0,962}$
rings5-regular	64-32-16	sigmoid	$F_1^{\text{macro}} = 0,955$

Widać, że „najlepsza aktywacja” zależy od zbioru: dla gładkiej wielomodalnej regresji sigmoid, dla funkcji schodkowej ReLU, dla klasyfikacji różnica jest w granicach szumu. W praktyce wybierałbym aktywację do kształtu funkcji celu, a nie do „domyślnej” zasady.



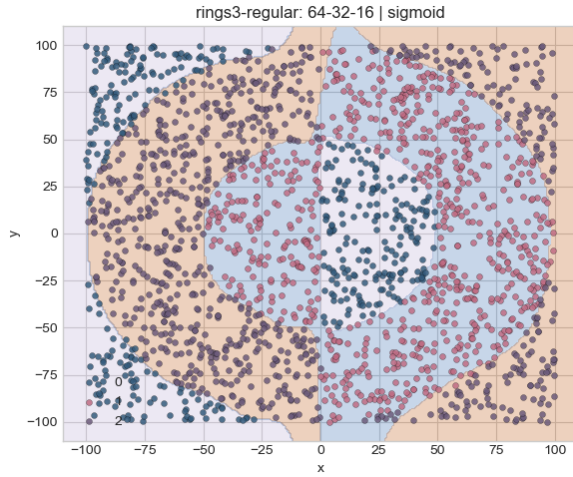
Rysunek 5: multimodal-large: porównanie najlepszych przebiegów dla badanych aktywacji.



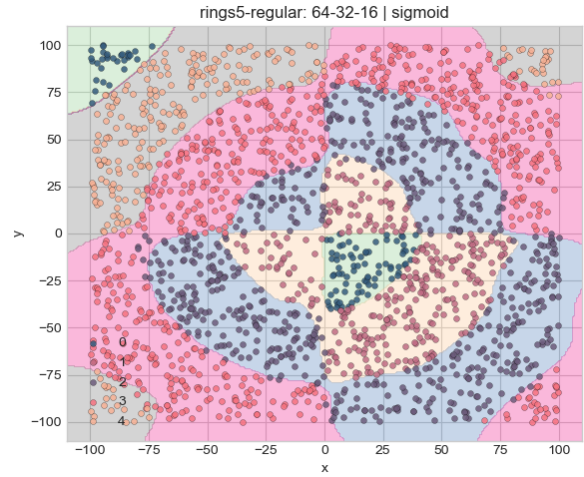
(a) multimodal-large: najlepszy wariant sigmoidalny.

(b) steps-large: ReLU vs sigmoid po przeniesieniu finalistów.

Rysunek 6: Regresja: wpływ aktywacji na kształt dopasowania.



(a) rings3-regular



(b) rings5-regular

Rysunek 7: Klasyfikacja: granice decyzyjne najlepszych modeli z NN5.

7 Przeuczenie i regularyzacja

W ostatniej części porównywałem cztery warianty na tej samej architekturze: baseline, ℓ_2 , *early stopping* i $\ell_2 + \text{early stopping}$. Architektura wybierałem wcześniej, w screeningu walidacyjnym.

Tabela 9: NN6: architektury wybrane po screeningu walidacyjnym.

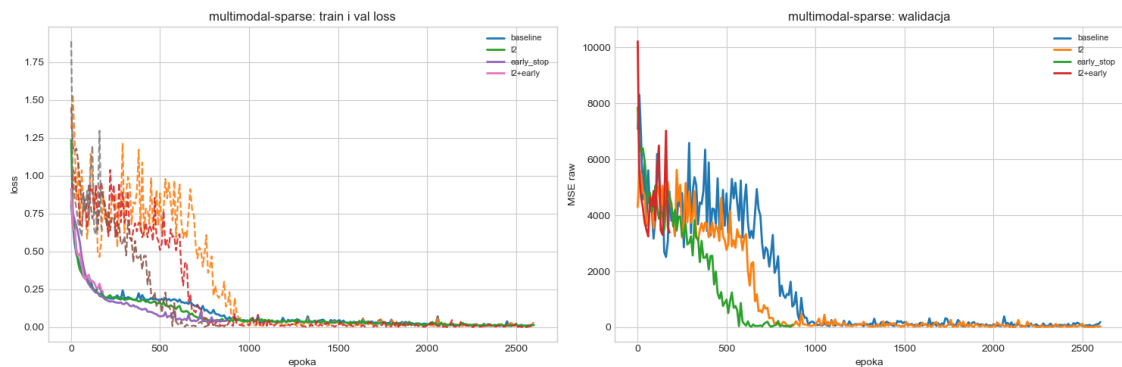
Zbiór	Architektura	Aktywacja	Wynik wal./test
multimodal-sparse	64-32-16	ReLU	$\text{MSE}_{\text{val}} = 12,90$, $\text{MSE}_{\text{test}} = 225,33$
rings5-sparse	64-32	tanh	$F_1^{\text{macro}}_{\text{val}} = 0,899$, $F_1^{\text{macro}}_{\text{test}} = 0,761$
rings3-balance	64-32-16	ReLU	$F_1^{\text{macro}}_{\text{val}} = 0,963$, $F_1^{\text{macro}}_{\text{test}} = 0,882$
xor3-balance	32-32-16	ReLU	$F_1^{\text{macro}}_{\text{val}} = 0,975$, $F_1^{\text{macro}}_{\text{test}} = 0,910$

Tabela 10: NN6: porównanie metod przeciwdziałania przeuczeniu.

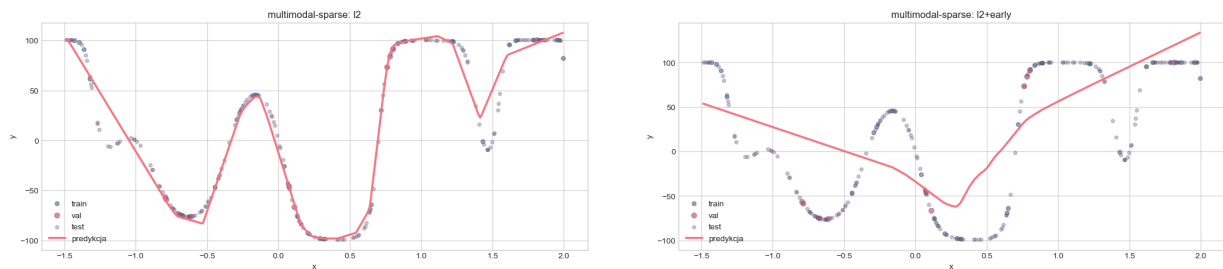
Zbiór	Wariant	λ	Stop	Best ep.	Wynik testowy
multimodal-sparse	baseline	0	2600	2260	$\text{MSE} = 270,085$
multimodal-sparse	ℓ_2	0,0020	2600	2110	$\text{MSE} = \mathbf{190,245}$
multimodal-sparse	early stop	0	870	750	$\text{MSE} = 341,770$
multimodal-sparse	$\ell_2 + \text{early}$	0,0020	180	60	$\text{MSE} = 2795,322$
rings5-sparse	baseline	0	2600	1120	$F_1^{\text{macro}} = 0,801$
rings5-sparse	ℓ_2	0,0012	2600	640	$F_1^{\text{macro}} = \mathbf{0,808}$
rings5-sparse	early stop	0	320	180	$F_1^{\text{macro}} = 0,664$
rings5-sparse	$\ell_2 + \text{early}$	0,0012	380	240	$F_1^{\text{macro}} = 0,752$
rings3-balance	baseline	0	2200	800	$F_1^{\text{macro}} = \mathbf{0,894}$
rings3-balance	ℓ_2	0,0008	2200	1140	$F_1^{\text{macro}} = 0,891$
rings3-balance	early stop	0	280	160	$F_1^{\text{macro}} = 0,869$
rings3-balance	$\ell_2 + \text{early}$	0,0008	420	300	$F_1^{\text{macro}} = 0,879$
xor3-balance	baseline	0	2200	120	$F_1^{\text{macro}} = 0,865$
xor3-balance	ℓ_2	0,0008	2200	1460	$F_1^{\text{macro}} = \mathbf{0,892}$
xor3-balance	early stop	0	360	240	$F_1^{\text{macro}} = 0,846$
xor3-balance	$\ell_2 + \text{early}$	0,0008	360	240	$F_1^{\text{macro}} = 0,828$

ℓ_2 wychodzi stabilnie: w trzech zbiorach poprawia wynik testowy, w czwartym nie psuje. Działa przewidywalnie, bo po prostu ogranicza amplitudę wag i nie ingeruje w budżet epok.

Z *early stopping* był większy problem. W moich konfiguracjach za często zatrzymywał uczenie na płaskim fragmencie krzywej walidacyjnej, który wyglądał na zakończenie treningu, ale w rzeczywistości był tylko powolną fazą spadku. Najmocniej widać to na **multimodal-sparse**, gdzie połączenie ℓ_2 z *early stopping* kończy się $\text{MSE} = 2795$. To nie jest „wada *early stoppingu* jako metody”, tylko efekt zbyt agresywnie ustawionych parametrów (krótka *patience*, ocena co zbyt mało epok). Bez dodatkowego strojenia cierpliwości *early stopping* dla tych zbiorów po prostu nie był bezpiecznym przełącznikiem.



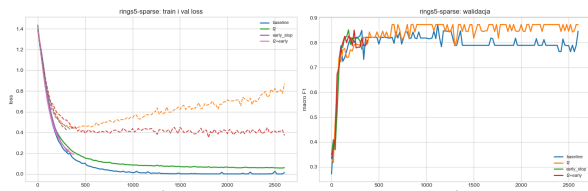
Rysunek 8: **multimodal-sparse**: krzywe uczenia i walidacji dla czterech wariantów.



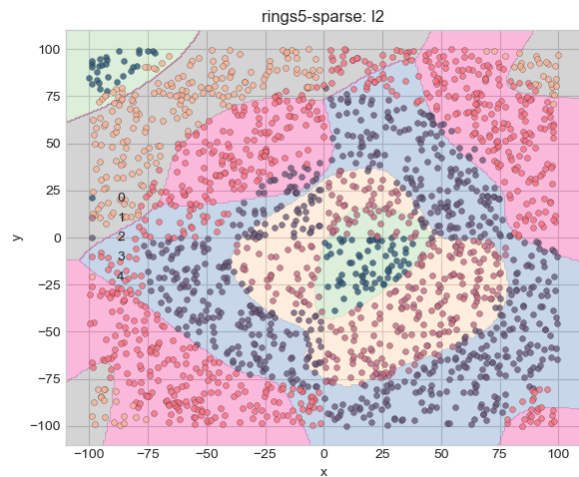
(a) ℓ_2 : najlepszy wariant regresyjny.

(b) $\ell_2 + \text{early stopping}$: wyraźnie niedouczony.

Rysunek 9: Porównanie jakości dopasowania na **multimodal-sparse**.

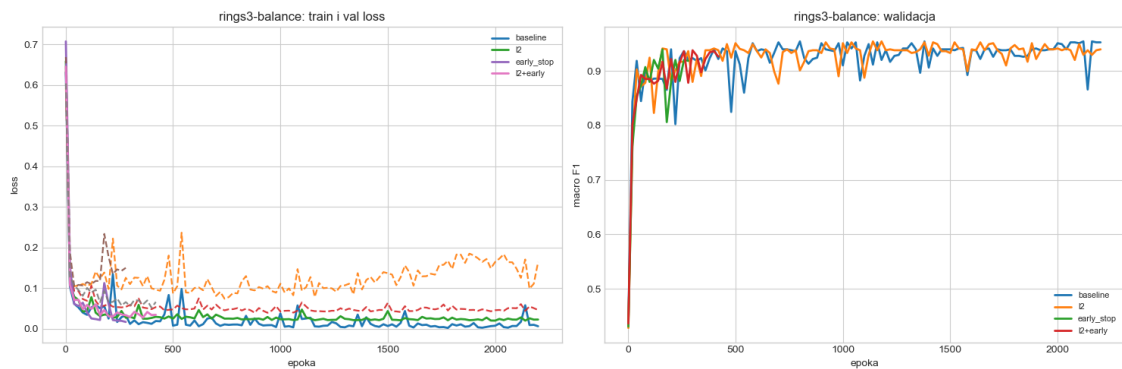


(a) rings5-sparse: przebiegi walidacyjne.

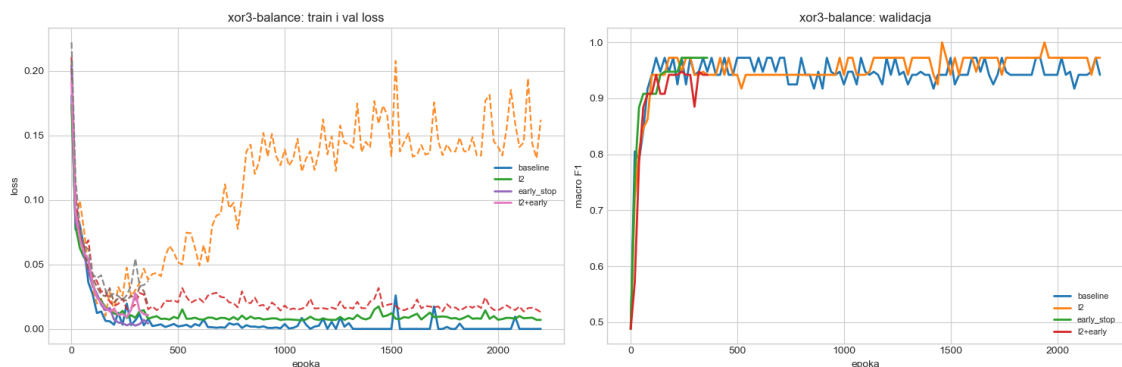


(b) rings5-sparse: granica decyzyjna wariantu ℓ_2 .

Rysunek 10: Dla rzadkich danych klasyfikacyjnych ℓ_2 wygładza granicę i daje najlepszy wynik testowy.



Rysunek 11: rings3-balance: wszystkie cztery warianty zbiegają do podobnego poziomu, baseline nieznacznie wygrywa na teście – przy zbalansowanych danych regularyzacja niewiele zmienia.



Rysunek 12: xor3-balance: ℓ_2 przesunęła najlepszą epokę z okolic 120 do 1460, opóźniając moment przeuczenia.

8 Wnioski

Z całego cyklu laboratoriów wynika kilka rzeczy, których nie widziałem, zanim zacząłem te eksperymenty. Po pierwsze, zachowanie MLP zależy bardziej od interakcji hiperparametrów niż

od pojedynczego wyboru. Sama liczba neuronów niewiele mówi, jeśli nie wiem, jak głęboka jest sieć i jaką aktywację dostaje. W NN1 1-5-1 było 1-10-1 na `square-simple`, a w NN3 ta sama logika „więcej = lepiej” też zawodziła.

Po drugie, przewaga konkretnej metody zwykle jest silnie zależna od zbioru. RMSProp jest najłatwiejszy do strojenia na stromej regresji (`square-large`, `steps-large`), ale Momentum wygrywa na gładkiej, wielomodalnej. Softmax jest lepszy bazowo, ale po dostrojeniu sigmoidy na `xor3` go dogania. ℓ_2 działa przewidywalnie prawie wszędzie, *early stopping* tylko przy starannie dobranej *patience*.

Po trzecie – najważniejsze z perspektywy implementacyjnej – zgodność z PyTorchem w NN1 i progów z polecenia w NN2 i NN3 pokazuje, że sam kod *jest* poprawny. Cała reszta to już strojenie.